```
1  ##############################################################################
2  # @file password_file.cpp
3  # @author Robert Myers Jr.
4  # @version V1.0
5  # @brief CMakeList file to compile Lab 1
6  ##############################################################################
7  cmake_minimum_required(VERSION 3.15)
8
9  project(Lab2 LANGUAGES CXX)
10
11 # Use C++ 20 features
12 set(CMAKE_CXX_STANDARD 20)
13 set(CMAKE_CXX_STANDARD_REQUIRED True)
14
15 # Generates a compile commands file for LSP
16 set(CMAKE_EXPORT_COMPILE_COMMANDS True)
17
18 add_executable(client
19     client.cpp
20     bank_service.cpp
21     tools/evp.cpp
22     lab2_client.cpp
23     user_input.cpp
24 )
25
26 add_executable(server
27     bank_service.cpp
28     tools/evp.cpp
29     lab2_server.cpp
30     server.cpp
31 )
32
33 # Link OpenSSL
34 find_package(OpenSSL REQUIRED)
35 target_link_libraries(client PRIVATE OpenSSL::Crypto)
36 target_link_libraries(server PRIVATE OpenSSL::Crypto)
37 target_include_directories(client PRIVATE .)
38 target_include_directories(server PRIVATE .)
```

```cpp
/**
 ********************************************************************************
 * @file bank_service.hpp
 * @author Robert Myers Jr.
 * @version V1.0
 * @brief Implementation of the BankService class. See header for more information.
 ********************************************************************************
 */
#include "bank_service.hpp"
#include "client.hpp"
#include "srp_utils.hpp"
#include "tools/evp.hpp"
#include "user_input.hpp"
#include <bit>
#include <cctype>
#include <chrono>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <optional>
#include <sstream>
#include <stdexcept>
#include <string>
#include <string_view>
#include <strings.h>
#include <sys/socket.h>
#include <sys/unistd.h>
#include <thread>
#include <vector>
#include <format>
#include <regex>

namespace {
    constexpr int b = 7;
    constexpr int max = 256;
    constexpr std::string_view TRANSCRIPT_FILE = "transcript_log.txt";
    bool convert_string(std::string_view floating_point_number, double & output) {
        for(int i = 0; i < floating_point_number.length(); i++) {
            if(!std::isdigit(floating_point_number[i]) && floating_point_number[i] != '.') {
                return false;
            }
        }
        try {
            output = std::stod(floating_point_number.data());
            return true;
        } catch (const std::invalid_argument& e) {
            return false;
        } catch (const std::out_of_range& e) {
            return false;
        }
    }
    double truncate(double floating_point_number) {
        return std::trunc(floating_point_number*100.00)/100.0;
    }
}

void BankService::send_message(std::string message, std::optional<uint64_t> key_mask){
    // std::cout << "Sending over " <<message << "\n";
    auto encrypted_message = quick_encrypt(message, key_mask);
    sendto(client_socket_, encrypted_message.data(), encrypted_message.size(), 0, (struct sockaddr *)&client_addr, sizeof(*client_addr));
}

BankService::BankService(
        int client_socket,
```

```
67          int account_number,
68          std::string salt,
69          std::string verifier) {
70      client_socket_ = client_socket;
71      account_number_ = account_number;
72      if(write_srp_information(std::to_string(account_number), salt, verifier)) {
73          std::cout << "Account created!" << std::endl;
74      } else {
75          send_message("ERROR! Account already exists!");
76          throw std::runtime_error("Account already exists!");
77      }
78  }
79
80  BankService::BankService(int client_socket,
81          int account_number,
82          int A):
83      client_socket_(client_socket),
84      account_number_(account_number),
85      balance(1000.0) {
86          // Get account balance if possible
87          std::ifstream transcript_file(TRANSCRIPT_FILE.data());
88
89          std::stringstream buffer;
90          buffer << transcript_file.rdbuf();
91          auto transcation_log = buffer.str();
92
93          auto line_to_find = std::format("- Account: {}", account_number_);
94
95          size_t found_pos = transcation_log.rfind(line_to_find);
96
97          // File is empty let's not try to read
98          if (found_pos != std::string::npos) {
99              size_t lineStart = transcation_log.rfind('\n', found_pos);
100
101             // Move past the newline character
102             lineStart++;
103
104             size_t lineEnd = transcation_log.find('\n', found_pos);
105
106             std::string lastLine = transcation_log.substr(lineStart, lineEnd - l
    ineStart);
107
108             size_t last_space_pos = lastLine.rfind(' ');
109
110             balance = std::stod(lastLine.substr(last_space_pos + 2));
111         }
112         // FETCH THE SRP INFORMATION AND START THE TRANSACTION
113         auto [I, S, V] = get_srp_information();
114
115         if(I == "") {
116             send_message("ERROR: Account not found. Closing Connection...");
117             throw std::runtime_error("Account not found");
118         }
119
120         // std::printf("I: %s S: %s V: %s\n", I.data(), S.data(), V.data());
121
122         auto B = SRP::generate_B(SRP::g, b, SRP::k, std::stoull(V), SRP::n);
123         auto message_for_client = std::format("{}:{}", S, B);
124         // std::cout << "Sending this to client" << message_for_client << "\n";
125         send_message(message_for_client);
126
127         auto U = SRP::hash_to_u64(std::format("{}:{}", A, B));
128
129         // std::cout << "U: " << U << "\n";
130
131         auto key = SRP::generate_key_server(A, std::stoull(V), U, b, SRP::n);
132         // std::cout << "key: " << key << "\n";
133         auto message = read_message(key);
134         // std::cout << "Message received: " << message << std::endl;
135         if(message == "Hi, I am the client") {
136             std::cout << "Client authenticated\n";
```

```
137                 send_message("Hi, I am the server", std::make_optional(key));
138             } else {
139                 std::cout << "Client authentication failed!\n";
140             }
141         }
142
143     bool BankService::run_bank_service(std::string input) {
144         std::string message_to_send;
145         if(input.starts_with("withdraw")) {
146             message_to_send = withdraw_command(input);
147         } else if(input.starts_with("deposit")) {
148             message_to_send = deposit_command(input);
149         } else if(input == "balance") {
150             message_to_send = balance_command();
151         } else if(input == "exit") {
152             exit_command();
153             return false;
154         } else {
155             message_to_send = "Error: invalid command\n";
156         }
157         message_to_send += "Enter Command (withdraw <amount>, deposit <amount>, balance, exit): ";
158         send_message(message_to_send);
159         return true;
160     }
161
162     void BankService::send_prompt()  {
163         std::string message_to_send = "Enter Command (withdraw <amount>, deposit <amount>, balance,
    exit): ";
164         send_message(message_to_send);
165     }
166
167     std::string BankService::deposit_command(std::string withdraw_command) {
168         std::stringstream stream(withdraw_command);
169
170         std::vector<std::string> words;
171         std::string word;
172
173         while(stream >> word) {
174             words.push_back(word);
175         }
176         if(words.size() >= 3 || words.size() <= 1) {
177             return std::format("Error: expected 1 numeric argument for the deposit command but got {} argu
    ments\n", words.size() - 1);
178         }
179
180         double amount_to_deposit = 0;
181
182         // Verify the argument is valid
183         if(!convert_string(words.at(1), amount_to_deposit) && amount_to_deposit >= 0
    .0) {
184             return "Error: Amount specified is invalid\n";
185         }
186
187         auto truncated_amount_to_deposit = truncate(amount_to_deposit);
188
189         std::string message_to_send = "";
190         if(truncated_amount_to_deposit != amount_to_deposit) {
191             message_to_send = std::format("Truncating ${:.3f} to ${:.2f}\n", amount_to_deposit
    , truncated_amount_to_deposit);
192         }
193
194         balance += truncated_amount_to_deposit;
195         message_to_send += std::format("Depositing ${:.2f}. Balance: ${:.2f}\n", truncated_amoun
    t_to_deposit, balance);
196
197         write_deposit_to_transcript(truncated_amount_to_deposit);
198
199         return message_to_send;
200     }
201
202     std::string BankService::withdraw_command(std::string withdraw_command) {
```

```
203     std::stringstream stream(withdraw_command);
204
205     std::vector<std::string> words;
206     std::string word;
207
208     while(stream >> word) {
209         words.push_back(word);
210     }
211     if(words.size() >= 3 || words.size() <= 1) {
212         return std::format("Error: expected 1 numeric argument for the withdraw command but got {} arguments\n", words.size() - 1);
213     }
214
215     double amount_to_withdraw = 0;
216
217     // Verify the amount is a floating_point_number and is positive
218     if(!convert_string(words.at(1), amount_to_withdraw) && amount_to_withdraw >= 0.0) {
219         return "Error: Amount specified is invalid\n";
220     }
221
222     auto truncated_amount_to_withdraw = truncate(amount_to_withdraw);
223
224     // Don't take out more money than whats in the account
225     if(truncated_amount_to_withdraw > balance) {
226         return std::format("Error: can't withdraw over the current balance of {:.2f}\n", balance);
227     }
228
229     std::string message_to_send = "";
230
231     if(truncated_amount_to_withdraw != amount_to_withdraw) {
232         message_to_send = std::format("Truncating ${:.3f} to ${:.2f}\n", amount_to_withdraw, truncated_amount_to_withdraw);
233     }
234
235     balance -= truncated_amount_to_withdraw;
236
237     write_withdraw_to_transcript(truncated_amount_to_withdraw);
238     return message_to_send + std::format("Withdrawing ${:.2f}. Balance: ${:.2f}\n", truncated_amount_to_withdraw, balance);
239 }
240
241 std::string BankService::balance_command() {
242     return std::format("Current balance ${:.2f}\n", balance);
243 }
244
245 void BankService::exit_command() {
246     std::string message_to_send = "Exiting...\n";
247     send_message(message_to_send);
248     std::cout << "Exit command received. Exiting...\n";
249     close(client_socket_);
250     exit(0);
251 }
252
253 std::string get_time() {
254     auto now = std::chrono::system_clock::now();
255
256     auto local_now = std::chrono::zoned_time{std::chrono::current_zone(), now};
257
258     auto seconds_only = std::chrono::floor<std::chrono::seconds>(local_now.get_local_time());
259
260     return std::format("{:%Y-%m-%d %H:%M:%S}\n", seconds_only);
261 }
262 void BankService::write_withdraw_to_transcript(double withdraw_amount) {
263     std::ofstream transcript_file(TRANSCRIPT_FILE.data(), std::ios_base::app);
264
265     auto formatted_time = get_time();
266
267     std::string formatted_deposit = std::format("- Account: {} - Withdraw of ${:.2f} successful. New balance : ${:.2f}\n", account_number_, withdraw_amount, balance);
```

```
268
269     transcript_file << formatted_time;
270     transcript_file << formatted_deposit;
271 }
272 void BankService::write_deposit_to_transcript(double deposit_amount) {
273     std::ofstream transcript_file(TRANSCRIPT_FILE.data(), std::ios_base::app);
274
275     auto formatted_time = get_time();
276
277     std::string formatted_deposit = std::format("– Account: {} – Deposit of ${:.2f} successful
    . New balance : ${:.2f}\n", account_number_, deposit_amount, balance);
278
279     transcript_file << formatted_time;
280     transcript_file << formatted_deposit;
281
282 }
283
284 std::string BankService::read_message(std::optional<uint64_t> key_mask) {
285     bzero(message_in_, max);
286     int len = recvfrom(client_socket_, message_in_, max, 0, NULL, NULL);
287     if (len == 0)
288     {
289         return "";
290     }
291     auto message_recevied = std::vector<unsigned char>(message_in_,
292             message_in_ + len);
293     return std::string(quick_decrypt(message_recevied, key_mask));
294 }
295
296 bool BankService::write_srp_information(std::string_view identify, std::string_v
    iew salt, std::string_view verifier) {
297     auto [I, S, V] = get_srp_information();
298     if(I != "") {
299         return false;
300     }
301     std::ofstream srp_file("srp_information.txt", std::ios_base::app);
302
303     auto formatted_time = get_time();
304
305     std::string formatted_srp_information = std::format("– Account: {} – identify: {} salt:
    {} verifier: {}\n", account_number_, identify, salt, verifier);
306     srp_file << formatted_srp_information;
307     return true;
308 }
309
310 std::tuple<std::string, std::string, std::string> BankService::get_srp_informati
    on() {
311     std::ifstream srp_file("srp_information.txt");
312
313     std::stringstream buffer;
314     buffer << srp_file.rdbuf();
315     auto srp_log = buffer.str();
316
317     auto line_to_find = std::format("– Account: {}", account_number_);
318
319     size_t found_pos = srp_log.rfind(line_to_find);
320
321     // File is empty let's not try to read
322     if (found_pos == std::string::npos) {
323         return std::make_tuple("", "", "");
324     } else {
325         // Authorization step
326     }
327
328     size_t lineStart = srp_log.rfind('\n', found_pos);
329
330     // Move past the newline character
331     lineStart++;
332
333     size_t lineEnd = srp_log.find('\n', found_pos);
334
```

```
335     std::string lastLine = srp_log.substr(lineStart, lineEnd - lineStart);
336     std::string reg_str = std::format("identify: {} salt: (\\d+) verifier: (\\d+)", account_numb
    er_);
337     std::regex reg(reg_str);
338     std::smatch matches;
339 
340     if (std::regex_search(lastLine, matches, reg)) {
341         std::string identify = matches[1].str();
342         std::cout << "found " << identify << std::endl;
343         uint64_t salt = std::stoull(matches[1].str());
344         uint64_t verifier = std::stoull(matches[2].str());
345         return std::make_tuple(
346             identify,
347             std::to_string(salt),
348             std::to_string(verifier)
349         );
350     }
351 
352 
353     return std::make_tuple("", "", "");
354 }
```

```cpp
#pragma once
/**
*****************************************************************************
* @file bank_service.hpp
* @author Robert Myers Jr.
* @version V1.0
* @brief BankService that handles deposits and withdraws for a single account.
*****************************************************************************
*/

#include <cstdint>
#include <netinet/in.h>
#include <optional>
#include <string>
#include <string_view>
#include <tuple>


class BankService {
    public:
        BankService(int client_socket,
                int account_number,
                int A = 0);
        BankService(int client_socket,
                int account_number,
                std::string salt,
                std::string verifier);
        void send_prompt();
        bool run_bank_service(std::string command);
    private:
        int client_socket_;
        int account_number_;
        double balance;
        struct sockaddr_in *client_addr;

        /**
         * @brief Sends a message to the client
         */
        void send_message(std::string message, std::optional<uint64_t> key_mask = std::nullopt);

        /**
         * @brief Writes successful deposit actions to the transaction log
         */
        void write_deposit_to_transcript(double deposit_amount);
        /**
         * @brief Writes successful withdraw actions to the transaction log
         */
        void write_withdraw_to_transcript(double withdraw_amount);

        /**
         * @brief Parse and executes a withdraw command. Returns a message indicating what happened
         *
         * @param withdraw_command The command itself
         */
        std::string withdraw_command(std::string withdraw_command);

        /**
         * @brief Parse and executes a deposit command. Returns a message indicating what happened
         *
         * @param deposit_command The command itself
         */
        std::string deposit_command(std::string deposit_command);

        bool write_srp_information(std::string_view identify, std::string_view salt, std::string_view verifier);

        std::tuple<std::string, std::string, std::string> get_srp_information();

```

```
68          /**
69           * @brief Executes the balance command
70           */
71          std::string balance_command();
72
73          /**
74           * @brief Executes the exit command
75           */
76          void exit_command();
77
78          std::string read_message(std::optional<uint64_t> key_mask = std::nullopt
    );
79          char message_in_[256];
80  };
```

```cpp
/**
 ********************************************************************************
 * @file Server.cpp
 * @author Robert Myers Jr.
 * @version V1.0
 * @brief Implementation of the Server Class. See header for more informating
 ********************************************************************************
 */
#include "server.hpp"
#include "bank_service.hpp"
#include "tools/evp.hpp"

#include <algorithm>
#include <cstdint>
#include <exception>
#include <iostream>
#include <iterator>
#include <sstream>
#include <string>
#include <strings.h>
#include <sys/types.h>
#include <sys/socket.h>

#include <arpa/inet.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <sys/unistd.h>
#include <vector>

constexpr int max = 256;
constexpr int port = 1234;

Server::Server() {
    sock_ = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (sock_ < 0)
    {
        std::cout << "socket call failed\n";
    }
    int yes = 1;
    setsockopt(sock_,SOL_SOCKET,SO_REUSEADDR,&yes,sizeof(int));

    server_addr_.sin_family = AF_INET;
    //inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
    server_addr_.sin_addr.s_addr = htonl(INADDR_ANY);
    server_addr_.sin_port = htons(port);

    printf("bind the socket: ");
    int r = bind(sock_, (struct sockaddr*)&server_addr_, sizeof(server_addr_));
    if (r<0)
    {
        printf("bind failed\n");
    }
    printf("server is listening ...\n");
    listen(sock_, 5); // queue length of 5
    printf("server init done\n");
}

std::string Server::read_message() {
    bzero(message_in_, max);
    int len = recvfrom(client_sock_, message_in_, max, 0, NULL, NULL);
    if (len == 0)
    {
        return "";
    }
    auto message_recevied = std::vector<unsigned char>(message_in_,
            message_in_ + len);
    return std::string(quick_decrypt(message_recevied));
}

std::string Server::send_message(std::string message){
    std::cout << "sending over " << message << std::endl;
```

```
72      auto encrypted_message = quick_encrypt(message);
73      sendto(client_sock_, encrypted_message.data(), encrypted_message.size(), 0,
    (struct sockaddr *)&client_addr_, sizeof(client_addr_));
74
75      bzero(message_in_, max);
76      int n = recvfrom(client_sock_, message_in_, max, 0, NULL, NULL);
77
78      auto message_received = std::vector<unsigned char>(message_in_, message_in_
    + n);
79
80      auto received = quick_decrypt(message_received);
81
82      return received;
83  }
84
85  int Server::get_account_number(std::string account_number_given) {
86      for(int i = 0; i < account_number_given.length(); i++) {
87          char letter = account_number_given.c_str()[i];
88          if(!isdigit(letter)) {
89              return -1;
90          }
91      }
92      try {
93          auto account_number = std::stoi(account_number_given);
94          if(account_number < 0) {
95              return -1;
96          }
97          return account_number;
98      } catch(const std::exception& e) {
99          return -1;
100     }
101 }
102
103 int Server::get_account_number() {
104     std::string message_to_send("Enter account number: ");
105
106     std::string account_number_given = send_message(message_to_send);
107
108     return get_account_number(account_number_given);
109 }
110
111 UserPath Server::get_prompt() {
112     std::string message_to_send("Welcome to RFM bank.\n'Registration' or 'Login': ");
113
114     std::string response = send_message(message_to_send);
115     std::transform(response.begin(), response.end(), response.begin(),
116             [](unsigned char c){ return std::tolower(c); });
117     if(response == "registration") {
118         return UserPath::Regristration;
119     } else if(response == "login") {
120         return UserPath::Login;
121     }
122     return UserPath::Error;
123
124 }
125
126 void Server::start_bank_server() {
127     while (1) //Try to accept a client request
128     {
129         printf("server: accepting new connection ...\n");
130
131         socklen_t len = sizeof(client_addr_);
132         client_sock_ = accept(sock_, (struct sockaddr *)&client_addr_, &len);
133
134         int account_number;
135         uint64_t A;
136
137         printf("server accepted a client connection from \n");
138         printf("Client: IP= %s port=%d \n", inet_ntoa(client_addr_.sin_addr), ntohs(cli
    ent_addr_.sin_port));
139         if (client_sock_ < 0)
```

```
140             {
141                 printf("server accept error \n");
142             }
143             auto action = get_prompt();
144             try {
145                 if(action == UserPath::Error) {
146                     std::cout << "Received Garbage Closing connection\n";
147                     send_message("Received Garbage Closing connection");
148                     close(client_sock_);
149                     continue;
150                 } else if(action == UserPath::Regristration) {
151                     auto register_string = send_message("register");
152                     std::stringstream ss(register_string);
153 
154                     std::string account_number_str;
155                     std::string salt;
156                     std::string verifier;
157 
158                     std::getline(ss, account_number_str, ':');
159                     std::getline(ss, salt, ':');
160                     std::getline(ss, verifier, ':');
161 
162                     account_number = get_account_number(account_number_str);
163 
164                     if(account_number == -1) {
165                         send_message("Error: invalid account number");
166                         close(client_sock_);
167                         continue;
168                     }
169 
170                     BankService(client_sock_, account_number, salt, verifier);
171                     send_message("You are now Registered!");
172                     continue;
173                 } else {
174                     auto register_string = send_message("login");
175                     std::stringstream ss(register_string);
176 
177                     std::string identify;
178                     std::string A_str;
179 
180                     std::getline(ss, identify, ':');
181                     std::getline(ss, A_str, ':');
182 
183                     account_number = get_account_number(identify);
184                     if(account_number == -1) {
185                         send_message("Error: invalid account number");
186                         close(client_sock_);
187                         continue;
188                     }
189                     A = std::stoull(A_str);
190                 }
191 
192                 BankService bank_service(client_sock_, account_number, A);
193                 bank_service.send_prompt();
194                 while(1)
195                 {
196                     auto received_message = read_message();
197                     if (received_message == "")
198                     {
199                         printf("server client died, server loops\n");
200                         close(client_sock_);
201                         break;
202                     }
203                     if(!bank_service.run_bank_service(received_message)) {
204                         break;
205                     }
206                 }
207             } catch (const std::runtime_error& e) {
208                 close(client_sock_);
209             }
210         }
```

```
211
212 }
```

```cpp
#pragma once
/**
*****************************************************************************
* @file Server.hpp
* @author Robert Myers Jr.
* @version V1.0
* @brief A server class made to make communicating with clients easier. Also runs the bank service.
*****************************************************************************
*/
#include <string>

#include <sys/types.h>
#include <sys/socket.h>

#include <arpa/inet.h>
#include <netinet/in.h>
#include <netinet/ip.h>

enum class UserPath { Login, Regristration, Error };
constexpr int buflen = 256;
class Server {
    public:
        Server();

        /**
         * @brief Starts the bank server and receiving messages. The program control is transferred to this function and will not return.
         */
        void start_bank_server();
    private:
        /**
         * @brief Sends a message to a connected client
         */
        std::string send_message(std::string message);

        /**
         * @brief Reads a message from a connected client
         */
        std::string read_message();

        /**
         * @brief Gets the account number
         */
        int get_account_number();

        /**
         * @brief Gets the account number
         */
        int get_account_number(std::string account_number_given);

        /**
         * Gets the prompt to login in or registration
         */
        UserPath get_prompt();

        int sock_;
        int client_sock_;

        std::string message_out_;
        char message_in_[buflen];

        struct sockaddr_in server_addr_;
        struct sockaddr_in client_addr_;
};
```

```cpp
#ifndef SRP_PROTOCOL_HPP
#define SRP_PROTOCOL_HPP

#include <string>
#include <string_view>
#include <cstdint>
#include <functional>
#include <format>

namespace SRP {
    constexpr uint64_t n = 53;
    constexpr uint64_t g = 2;
    constexpr uint64_t k = 11;

    /**
     * @brief Performs modular exponentiation (base^exp % mod).
     * Prevents overflow by using __uint128_t.
     */
    inline uint64_t mod_pow(uint64_t base, uint64_t exp, uint64_t mod) {
        uint64_t res = 1;
        base %= mod;
        while (exp > 0) {
            if (exp % 2 == 1) res = (static_cast<__uint128_t>(res) * base) % mod;
            base = (static_cast<__uint128_t>(base) * base) % mod;
            exp /= 2;
        }
        return res;
    }

    /**
     * @brief Simple hash wrapper.
     */
    inline uint64_t hash_to_u64(const std::string& input) {
        std::hash<std::string> hasher;
        return static_cast<uint64_t>(hasher(input));
    }


    inline uint64_t create_x(std::string_view salt, std::string_view identity, std::string_view password) {
        std::string hash_input = std::format("{}:{}:{}", salt, identity, password);
        uint64_t x = hash_to_u64(hash_input);
        return x;
    }

    /**
     * @brief Generates the password verifier (v)
     */
    inline uint64_t create_verifier(uint64_t x) {
        return mod_pow(g,x,n);
    }

    /**
     * @brief Client Public Value: A = g^a % n
     */
    inline uint64_t generate_A(uint64_t g, uint64_t a, uint64_t n) {
        return mod_pow(g, a, n);
    }

    /**
     * @brief Server Public Value: B = kv + g^b % n
     */
    inline uint64_t generate_B(uint64_t g, uint64_t b, uint64_t k, uint64_t v, uint64_t n) {
        uint64_t term2 = mod_pow(g, b, n);
        return ((k * v) % n + term2) % n;
    }

    /**
```

```
68          * @brief Client Shared Secret: S = (B - kg^x)^(a + ux) % n
69          */
70         inline uint64_t generate_key_client(uint64_t x, uint64_t B, uint64_t k, uint
    64_t g, uint64_t a, uint64_t u, uint64_t n) {
71             uint64_t g_x = mod_pow(g, x, n);
72             uint64_t k_gx = (static_cast<__uint128_t>(k) * g_x) % n;
73
74             // Ensure base remains positive before modulo
75             uint64_t base = (B >= k_gx) ? (B - k_gx) : (n - (k_gx - B) % n);
76             uint64_t exp = (a + (static_cast<__uint128_t>(u) * x) % (n - 1)) % (n -
    1);
77
78             return mod_pow(base, exp, n);
79         }
80
81         /**
82          * @brief Server Shared Secret: S = (A * v^u)^b % n
83          */
84         inline uint64_t generate_key_server(uint64_t A, uint64_t v, uint64_t u, uint
    64_t b, uint64_t n) {
85             uint64_t v_u = mod_pow(v, u % (n - 1), n);
86
87             uint64_t base = (static_cast<__uint128_t>(A) * v_u) % n;
88
89             return mod_pow(base, b, n);
90         }
91     }
92
93     #endif // SRP_PROTOCOL_HPP
```

```cpp
/**
*****************************************************************************
* @file client.cpp
* @author Robert Myers Jr.
* @version V1.0
* @brief Implementation of the client class. See header for more information
*****************************************************************************
*/
#include "client.hpp"
#include "tools/evp.hpp"

#include <cstdint>
#include <iostream>
#include <openssl/evp.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <openssl/rand.h>
#include <string.h>
#include <strings.h>
#include <unistd.h>

constexpr int max = 1024;
constexpr int port = 1234;
constexpr const char * server_ip = "127.0.0.1";

Client::Client() {
  sock_ = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
  server_addr_.sin_family = AF_INET;
  inet_pton(AF_INET, server_ip, &server_addr_.sin_addr);

  server_addr_.sin_port = htons(port);

  r_ = connect(sock_, (struct sockaddr*)&server_addr_, sizeof(server_addr_));
}

std::string Client::send_message(std::string message) {
    write(sock_, message.c_str(), max);

    bzero(message_in_, max);
    read(sock_, message_in_, max);
    return std::string(message_in_);
}

std::string Client::read_message() {
    bzero(message_in_, max);
    read(sock_, message_in_, max);
    return std::string(message_in_);
}

std::string Client::read_message_aes() {
    bzero(message_in_, max);
    int len = read(sock_, message_in_, max);
    auto message_recevied = std::vector<unsigned char>(message_in_,
            message_in_ + len);
    return std::string(quick_decrypt(message_recevied));
}

std::string Client::send_message_aes(std::string message, std::optional<uint64_t
> key_mask) {
    auto encrypted_message = quick_encrypt(message, key_mask);
    write(sock_, encrypted_message.data(), encrypted_message.size());
    int n = read(sock_, message_in_, max);
    auto message_recevied = std::vector<unsigned char>(message_in_,
            message_in_ + n);
    return quick_decrypt(message_recevied, key_mask);
}
```

```cpp
#pragma once
/**
*****************************************************************************
* @file client.hpp
* @author Robert Myers Jr.
* @version V1.0
* @brief A client class made to make communicating with the server easier
*****************************************************************************
*/

#include <cstdint>
#include <optional>
#include <string>
#include <sys/socket.h>
#include <netinet/in.h>

constexpr int buflen = 1024;
class Client {
    public:
        Client();
        /**
         * @brief Sends and receives a message
         *
         * @param message The message received
         */
        std::string send_message(std::string message);

        /**
         * @brief Reads a message
         */
        std::string read_message();

        /**
         * @brief Reads a AES encypted message. Will return the decrypted messag
e
         */
        std::string read_message_aes();

        /**
         * @brief Sends and receives a message with AES encryption. Will returne
d a decrypted string
         *
         * @param message The message being sent
         */
        std::string send_message_aes(std::string message, std::optional<uint64_t
> key_mask = std::nullopt);
    private:
        int sock_;
        int r_;
        struct sockaddr_in server_addr_;
        char message_out_[buflen];
        char message_in_[buflen];
};
```

```cpp
#include "client.hpp"
#include "user_input.hpp"
#include <cctype>
#include <cstdio>
#include <cstdlib>
#include <format>
#include <iostream>
#include <optional>
#include <sstream>
#include <string>
#include <string_view>
#include <srp_utils.hpp>

namespace {
    constexpr int a = 5;
    constexpr int v = 7;
    constexpr int s = 7;
}
void login_procedure(Client& client) {
    auto A = SRP::generate_A(SRP::g, a, SRP::n);

    std::cout << "Input Account Number: ";
    std::string identify;
    std::getline(std::cin, identify);

    std::cout << "Input Password: ";
    std::string password;
    std::getline(std::cin, password);

    auto challenge = client.send_message_aes(
            std::format("{}:{}", identify, A)
    );
    if (challenge == "Error: invalid account number") {
        std::cout << challenge << std::endl;
        std::cout << "Exiting..."<< std::endl;
        exit(0);
    } else if(challenge == "ERROR: Account not found. Closing Connection...") {
        std::cout << challenge << std::endl;
        std::cout << "Exiting..."<< std::endl;
        exit(0);
    }

    std::stringstream ss(challenge);

    std::string salt;
    std::string B;

    std::getline(ss, salt, ':');
    std::getline(ss, B, ':');

    auto u = SRP::hash_to_u64(std::format("{}:{}",A,B));
    //std::cout << "salt: " << salt << "\n";
    // std::cout << "U: " << u << "\n";

    auto x = SRP::create_x(salt, identify, password);
    // std::cout << "X  loging " << x << std::endl;

    auto key_A = SRP::generate_key_client(x, std::stoull(B), SRP::k, SRP::g, a, u, SRP::n);

    // std::cout << "KEY:"<<key_A<<"\n";

    auto server_message = client.send_message_aes("Hi, I am the client", std::make_optional(key_A));
    // std::cout << "Server message" << server_message << "\n";
    if(server_message == "Hi, I am the server") {
        std::cout << "Server authenticated\n";
    } else {
        std::cout << "Server authentication failed!\n";
        std::cout << "Exiting...\n";
        exit(0);
```

```
70      }
71  }
72  void register_procedure(Client& client) {
73      std::cout << "Starting registration process.\n";
74      std::cout << "Input Account Number: ";
75      std::string identify;
76      std::getline(std::cin, identify);
77
78      std::cout << "Input Password: ";
79      std::string password;
80      std::getline(std::cin, password);
81      std::string salt = std::to_string(s);
82
83      auto x = SRP::create_x(
84              std::string_view(salt),
85              std::string_view(identify),
86              std::string_view(password)
87              );
88      //std::cout << "X  registartion" << x << std::endl;
89      auto verifier = SRP::create_verifier(x);
90      auto message = (client.send_message_aes(
91              std::format("{}:{}:{}", identify, salt, verifier)
92      ));
93      if (message == "Error: invalid account number") {
94          std::cout <<  message << std::endl;
95          std::cout << "Exiting..."<< std::endl;
96          exit(0);
97      }
98  }
99
100 int main() {
101     Client client;
102
103     std::string server_message;
104     //server_message = client.send_message(" ");
105     std::cout << "Connection established\n";
106     std::cout << client.read_message_aes();
107     while(1) {
108         server_message = client.send_message_aes(get_user_input(server_message))
    ;
109         if(server_message == "register") {
110             register_procedure(client);
111             std::cout << "Exiting program...\n";
112             exit(0);
113         } else if(server_message == "login") {
114             login_procedure(client);
115             server_message = client.read_message_aes();
116         } else if(server_message.find("Exiting") != std::string::npos) {
117             std::cout << server_message;
118             exit(0);
119         } else if(server_message == "Received Garbage Closing connection") {
120             std::cout << "Server Closed connection due to garbage\n";
121             exit(0);
122         } else {
123             std::cout << server_message;
124         }
125     }
126 }
```

```cpp

#include "server.hpp"
int main() {
    Server server;
    server.start_bank_server();
}
```

```cpp
/**
*****************************************************************************
* @file evp.cpp
* @author Robert Myers Jr.
* @version V1.0
* @brief Implementation of the evp functions. See header for more information
*****************************************************************************
*/
#include "evp.hpp"
#include <array>
#include <cstdint>
#include <cstring>
#include <iostream>
#include <numbers>
#include <openssl/evp.h>
#include <openssl/rand.h>
#include <optional>
#include <ostream>
#include <span>
#include <string>
#include <strings.h>
#include <vector>

constexpr int max = 1024;

int encrypt (const unsigned char *plaintext, int plaintext_len, unsigned char *key, unsigned char *iv, unsigned char *ciphertext) {
    EVP_CIPHER_CTX *ctx;
    int len;
    int ciphertext_len;

    if (!(ctx = EVP_CIPHER_CTX_new())) return -1;
    if (EVP_EncryptInit_ex(ctx, EVP_aes_256_cbc(), NULL, key, iv) != 1) return -1;
    if (EVP_EncryptUpdate (ctx, ciphertext, &len, plaintext, plaintext_len) != 1) return -1;

    ciphertext_len = len;

    if (EVP_EncryptFinal_ex(ctx, ciphertext + len, &len) != 1) return -1;

    ciphertext_len += len;
    EVP_CIPHER_CTX_free(ctx);

    return ciphertext_len;
}
int decrypt (const unsigned char *ciphertext, int ciphertext_len, unsigned char *key, unsigned char *iv, unsigned char *plaintext) {
    EVP_CIPHER_CTX *ctx;
    int len;
    int plaintext_len;

    if (!(ctx = EVP_CIPHER_CTX_new())) return -1;

    if (EVP_DecryptInit_ex(ctx, EVP_aes_256_cbc(), NULL, key, iv) != 1) return -1;

    if (EVP_DecryptUpdate (ctx, plaintext, &len, ciphertext, ciphertext_len) != 1) return -1;

    plaintext_len = len;

    if (EVP_DecryptFinal_ex(ctx, plaintext + len, &len) != 1) return -1;
    plaintext_len += len;

    EVP_CIPHER_CTX_free(ctx);
    return plaintext_len;
}
std::vector<unsigned char> quick_encrypt(std::string& plaintext, std::optional<uint64_t> key_mask) {
    unsigned char key_in_use[33];
```

```
65      strncpy((char*)key_in_use, (const char*)key, 33);
66      if(key_mask) {
67          ((uint64_t*)key_in_use)[0] = key_mask.value();
68      }
69      std::vector<unsigned char> buffer(EVP_MAX_IV_LENGTH + plaintext.length() + E
    VP_MAX_BLOCK_LENGTH);
70
71      RAND_bytes(buffer.data(), EVP_MAX_IV_LENGTH);
72
73      int cipher_len = encrypt(
74          (unsigned char*)plaintext.data(), (int)plaintext.length(),
75          (unsigned char*)key_in_use,
76          buffer.data(),
77          buffer.data() + EVP_MAX_IV_LENGTH
78      );
79
80      buffer.resize(EVP_MAX_IV_LENGTH + cipher_len);
81      return buffer;
82  }
83
84  std::string quick_decrypt(std::vector<unsigned char>& encrypted_message, std::op
    tional<uint64_t> key_mask) {
85      if (encrypted_message.size() < EVP_MAX_IV_LENGTH) return "";
86      unsigned char key_in_use[33];
87      strncpy((char*)key_in_use, (const char*)key, 33);
88      if(key_mask) {
89          ((uint64_t*)key_in_use)[0] = key_mask.value();
90      }
91
92      int ciphertext_len = (int)encrypted_message.size() - EVP_MAX_IV_LENGTH;
93      std::vector<unsigned char> plain_buf(ciphertext_len + EVP_MAX_BLOCK_LENGTH);
94
95      int plain_len = decrypt(
96          encrypted_message.data() + EVP_MAX_IV_LENGTH,
97          ciphertext_len,
98          (unsigned char*)key_in_use,
99          encrypted_message.data(),
100         plain_buf.data()
101     );
102
103     if (plain_len <= 0) return "DECRYPTION_FAILED";
104
105     return std::string((char*)plain_buf.data(), plain_len);
106 }
```

```cpp
#pragma once
/**
*****************************************************************************
* @file evp.hpp
* @author Robert Myers Jr.
* @version V1.0
* @brief evp functions to encrypt and decrypt
*****************************************************************************
*/

#include <cstdint>
#include <optional>
#include <span>
#include <string>
#include <vector>
const unsigned char key[33] = "01234567890123456789012345678901";

/**
 * @brief encrypts a message using aes
 *
 * @param plaintext Text being encrypted. Code was lifted and modified from lab example
 * @param plaintext_len the len
 * @param key The key to the message
 * @param iv The iv block
 * @param ciphertext The resulting ciphertext
 * @return
 */
int encrypt(const unsigned char *plaintext, int plaintext_len, unsigned char*key,
        unsigned char *iv, unsigned char *ciphertext);

/**
 * @brief encrypts a message using aes. Code was lifted and modified from lab example
 *
 * @param ciphertext Text being decrypted
 * @param plaintext_len the len
 * @param key The key to the message
 * @param iv The iv block
 * @param plaintext The resulting plain text
 * @return
 */
int decrypt(const unsigned char *ciphertext, int plaintext_len, unsigned char*key,
        unsigned char *iv, unsigned char *plaintext);

/**
 * @brief Helper method to quickly encrypt data
 */
std::vector<unsigned char> quick_encrypt(std::string& message, std::optional<uint64_t> key_mask = std::nullopt);


/**
 * @brief Helper method to quickly decrypt data
 */
std::string quick_decrypt(std::vector<unsigned char>& encrypted_message, std::optional<uint64_t> key_mask = std::nullopt);

```